INF6303 — Devoir 2

Partie recommandation : ALS implicite

Rapport compilé le 13 novembre 2026 à partir des sorties déjà produites. Le code **src/devoir2_spark.py** et les fichiers de **results/** n'ont pas été modifiés.

- Cours : INF6303
- Partie : C, recommandation de produits
- Moteur : Apache Spark 4.2.0, MLlib ALS, master **spark://spark-master:7077**
- Exécutions : 1 worker et 3 workers, 24 partitions
- Chiffres : **results/resultat_1_worker.json** et **results/resultat_3_workers.json**, champ **als**
- Exemple de produits : catégories et marques lues dans **data/catalogue.csv**
- Date de référence des achats : 1er novembre 2026
- Graine : 42

Synthèse

La configuration retenue est **A_rank10** : factorisation ALS implicite de rang 10, 10 itérations, régularisation 0,1, coefficient de confiance alpha 40.

Configuration	Recall@10	Clients avec au moins un succès	Part des 10 383 clients évalués
A_rank10	0,079953	2 592	24,96 %
B_rank20	0,070010	2 396	23,08 %
Popularité (référence)	0,015236	806	7,76 %

Le Recall@10 et le partage entraînement / test sont identiques avec 1 worker et avec 3 workers, jusqu'au sixième chiffre. Le rang 10 vaut 5,25 fois le recall de la popularité. Il dépasse le rang 20 de 0,009943 point, soit 14,2 % en relatif, et retrouve un achat masqué pour 196 clients de plus.

La partie ALS complète dure 99,994 s avec 1 worker et 105,779 s avec 3 workers. Le rapport des temps est 0,95 : sur ce volume, trois workers allongent le calcul.

1. Question traitée

La partie recommandation propose, pour chaque client, dix produits qu'il n'a pas encore achetés, puis mesure si cette liste retrouve des achats volontairement cachés.

Le signal est implicite. Le fichier de transactions ne contient pas de note. Le fait d'acheter un produit, et la quantité achetée, servent de trace de préférence.

Cette partie lit les transactions déjà nettoyées. Elle n'utilise ni le tableau RFM, ni les segments du K-Means.

2. Données qui entrent dans le modèle

Le nettoyage laisse 197 677 transactions, issues de 198 350 lignes brutes. Les 673 lignes retirées se répartissent en 200 quantités non positives, 125 prix non positifs et 348 doublons exacts. Aucun identifiant manquant, aucune date invalide ou postérieure au 1er novembre 2026, aucune transaction sans client connu, aucune transaction sans produit du catalogue.

À partir de ces transactions, la fonction **recommander** construit un triplet par couple distinct (client,

produit). On obtient :

Ensemble	Effectif
Triplets (client, produit)	184 962
Clients	12 000
Produits du catalogue	800
Transactions propres	197 677

Plusieurs transactions du même couple sont donc fondues en une seule ligne. La force d'interaction, appelée **strength**, est la somme des quantités, convertie en flottant.

3. Méthode

3.1 Des achats aux triplets

Pour chaque couple (**client_id**, **produit_id**), Spark calcule la somme des quantités. Deux achats du même produit renforcent le même signal. Le prix, la date et la région n'entrent pas dans ALS.

3.2 Identifiants entiers

ALS exige des identifiants entiers. Chaque client reçoit un **uid** et chaque produit un **iid**, par numérotation ordonnée (**row_number** sur l'identifiant texte). L'ordre est fixe d'une exécution à l'autre. Ces deux tables sont petites et sont diffusées (**broadcast**) au moment de la jointure, ce qui évite de redistribuer les 184 962 triplets.

3.3 Partage entraînement / test

Chaque couple est orienté par un hachage :

pmod(hash(client_id, produit_id, 42), 100)

Le couple va à l'entraînement si le résultat est strictement inférieur à 80, au test sinon. Un couple figure donc d'un seul côté. Le partage dépend de la graine et des identifiants, et il est le même quel que soit le nombre de workers ou l'ordre des partitions.

Ensemble	Effectif	Part
Triplets au total	184 962	100 %
Entraînement	148 083	80,06 %
Test avant filtre	36 879	19,94 %
Test retenu pour la mesure	36 869	19,93 %
Test retiré (démarrage à froid)	10	0,027 % du test

3.4 Démarrage à froid

Un client ou un produit absent de l'entraînement n'a pas de vecteur latent. Le test utilisé pour le recall ne garde que les triplets dont le client et le produit apparaissent dans l'entraînement. ALS est en plus lancé avec **coldStartStrategy = drop**.

Population	Effectif
Clients au total	12 000
Clients sans aucune interaction d'entraînement	2
Clients évalués	10 383
Clients avec historique d'entraînement et sans triplet de test retenu	1 615

Les 10 383 clients évalués représentent 86,53 % des clients. Chacun a en moyenne 3,55 produits dans le test retenu (36 869 / 10 383).

Les 1 615 clients restants, une fois les 2 clients sans historique mis à part, ont un passé d'achats et aucun triplet de test conservé. Le filtre de démarrage à froid ne retire que 10 triplets sur l'ensemble du test : la grande majorité de ces 1 615 clients a simplement vu tous ses couples partir dans l'entraînement. Ils sont hors du recall. Ils ne le font ni monter ni descendre.

3.5 Apprentissage ALS

Le modèle est un ALS à retours implicites, dans la lignée de Hu, Koren et Volinsky (2008). Dès qu'un couple est observé, la préférence vaut 1. La confiance accordée à cette observation croît avec la quantité :

c = 1 + alpha × strength, avec alpha = 40.

Les facteurs sont contraints à être non négatifs. Les deux configurations partagent tous les réglages sauf le nombre de facteurs.

Paramètre	A_rank10	B_rank20
rank	10	20
maxIter	10	10
regParam	0,1	0,1
alpha	40	40

Le code fixe aussi, pour les deux modèles, **implicitPrefs = true**, **nonnegative = true**, **coldStartStrategy = drop** et **seed = 42**. Ces quatre arguments sont dans l'appel ALS. Ils figurent ici parce qu'ils déterminent le résultat. Le dictionnaire **parametres** du JSON, lui, ne contient que rank, maxIter, regParam et alpha.

Seul le rang change. L'écart de recall entre A et B est donc lié au nombre de facteurs, avec une régularisation et un nombre d'itérations tenus fixes.

3.6 Liste de dix produits nouveaux

La mesure porte sur des produits que le client n'a pas achetés dans l'entraînement. Pour qu'il reste 10 produits après ce retrait, le modèle est d'abord interrogé sur un nombre plus grand de candidats : 10, plus le maximum de produits déjà vus par un client du test, sans dépasser le nombre de produits distincts de l'entraînement.

Les produits déjà vus sont ensuite enlevés par anti-jointure avec l'entraînement. Le rang final est le score décroissant, puis l'identifiant produit en cas d'égalité. On garde les 10 premiers.

Le score est le produit scalaire du vecteur client et du vecteur produit. C'est un score de confiance implicite. Il classe les produits d'un même client. Il n'est pas une probabilité, et deux clients ne se comparent pas par la valeur brute de ce score.

3.7 Recall@10

Pour chaque client évalué u :

Recall@10(u) = |Top 10(u) ∩ Test(u)| / |Test(u)|

Le Recall@10 publié est la moyenne de ces valeurs. Le rapport donne aussi le nombre de clients pour lesquels l'intersection contient au moins un produit.

Un recall de 0,08 signifie qu'en moyenne 8 % des produits masqués d'un client évalué se retrouvent dans ses 10 suggestions.

3.8 Référence de popularité

La même liste de 10 produits, avec le même retrait des produits déjà vus, est construite sans ALS. Le

classement est le nombre de clients distincts ayant acheté le produit dans l'entraînement. Cette référence situe le gain apporté par les facteurs latents par rapport à « proposer ce que le plus de gens achètent ».

4. Résultats

4.1 Qualité

Les métriques ci-dessous sont celles des deux fichiers JSON. Elles coïncident exactement.

Configuration	Recall@10	Clients avec au moins un succès	Part des clients évalués
A_rank10	0,079953	2 592	24,96 %
B_rank20	0,070010	2 396	23,08 %
Popularité	0,015236	806	7,76 %

Lecture :

- Environ un client évalué sur quatre (24,96 %) voit au moins un de ses achats masqués dans le top 10 du rang 10. Avec le rang 20, cette part est 23,08 %. Avec la popularité, elle tombe à 7,76 %.
- Le rang 10 retrouve donc un signal que la popularité laisse passer. Le facteur 5,25 entre les deux recall (0,079953 / 0,015236) vient de là : les facteurs latents rapprochent des clients qui se ressemblent, au-delà du seul succès commercial d'un produit.
- Passer de 10 à 20 facteurs, sans relever la régularisation ni le nombre d'itérations, baisse le recall. Sur 148 083 triplets d'entraînement, 12 000 clients et un catalogue de 800 produits, le rang 20 ajoute de la capacité. Avec les réglages tenus fixes, cette capacité se généralise moins bien que le rang 10.
- Le recall reste modeste dans l'absolu : 8 % des achats masqués. Chaque client évalué n'a en moyenne que 3,55 produits dans le test, et la liste n'en propose que 10, tirés d'un catalogue de 800 références. Le chiffre utile pour juger le modèle est l'écart à la popularité, obtenu dans le même protocole.

4.2 Durées

Le chronomètre de chaque configuration couvre l'apprentissage, la construction du top 10 et le calcul du recall. Le champ **als_seconds** couvre en plus les triplets, l'indexation, le partage et la référence de popularité.

Mesure	1 worker (s)	3 workers (s)
Configuration A, rang 10	41,851	44,886
Configuration B, rang 20	35,371	39,288
Partie ALS complète	99,994	105,779

Par différence, le hors-modèle de **recommander** (triplets, index, partage, popularité) représente 22,8 s avec 1 worker et 21,6 s avec 3 workers.

Dans les deux exécutions, le rang 20 est plus court que le rang 10. L'écart est reproductible sur ce cluster. Le choix de configuration s'appuie sur le recall, pas sur cette durée. La durée mesurée mélange l'apprentissage et l'évaluation, et les deux modèles s'enchaînent dans le même processus Spark.

Le rapport T(1 worker) / T(3 workers) sur la partie ALS complète vaut 99,994 / 105,779 = 0,95. Trois workers allongent cette partie d'environ 5,8 %. ALS enchaîne des itérations et des échanges entre les 24 partitions. Avec environ 148 000 triplets, un exécuteur d'un seul cœur et 1 100 Mo, le coût de coordination l'emporte sur le gain de parallélisme. La qualité, elle, est stable : le recall ne change pas.

5. Exemple de top 10

L'exemple provient de la meilleure configuration, A_rank10. Le code trie par **client_id** puis par rang, et garde les 30 premières lignes : les dix produits des trois premiers identifiants dans l'ordre lexicographique. Ce sont les clients 0, 1 et 10. C'est un échantillon d'identifiants, pas une sélection des meilleurs scores.

Les scores du JSON sont ceux de la colonne arrondie à quatre décimales. Le rapport les affiche à quatre décimales. Catégorie et marque viennent du catalogue. Le modèle ne les reçoit pas : elles servent uniquement à lire l'exemple.

Client 0

Les dix suggestions sont dans la catégorie Maison. Les dix marques sont distinctes. L'écart entre le premier et le dixième score est 0,014.

Rang	Produit	Catégorie	Marque	Score
1	752	Maison	M16	0,9507
2	736	Maison	M00	0,9476
3	320	Maison	M21	0,9431
4	600	Maison	M02	0,9423
5	640	Maison	M19	0,9404
6	544	Maison	M15	0,9392
7	712	Maison	M22	0,9377
8	152	Maison	M14	0,9370
9	112	Maison	M20	0,9366
10	776	Maison	M17	0,9364

Client 1

Les dix suggestions sont dans la catégorie Culture. Les dix marques sont distinctes. Les scores sont plus bas que ceux du client 0 (0,67 à 0,69) et restent serrés : 0,021 entre le premier et le dixième.

Rang	Produit	Catégorie	Marque	Score
1	499	Culture	M16	0,6920
2	395	Culture	M04	0,6887
3	611	Culture	M13	0,6840
4	275	Culture	M22	0,6810
5	347	Culture	M02	0,6784
6	603	Culture	M05	0,6782
7	107	Culture	M15	0,6758
8	539	Culture	M10	0,6734
9	435	Culture	M21	0,6731
10	43	Culture	M20	0,6711

Client 10

Les dix suggestions sont dans la catégorie Santé (valeur **Sante** dans le catalogue). La marque M16 revient trois fois (produits 614, 246 et 62). Les sept autres marques sont distinctes. L'écart de score entre le

premier et le dixième est 0,023.

Rang	Produit	Catégorie	Marque	Score
1	662	Santé	M18	0,8760
2	526	Santé	M20	0,8756
3	454	Santé	M17	0,8749
4	614	Santé	M16	0,8725
5	246	Santé	M16	0,8622
6	62	Santé	M16	0,8614
7	390	Santé	M22	0,8610
8	678	Santé	M11	0,8593
9	470	Santé	M10	0,8568
10	334	Santé	M12	0,8534

Lecture de l'exemple

Pour ces trois clients, le top 10 reste à l'intérieur d'une seule catégorie, alors que la catégorie n'est pas une entrée du modèle. Elle ressort parce que des clients qui achètent les mêmes produits se retrouvent proches dans l'espace latent, et que ces produits partagent une catégorie dans le catalogue.

Les marques, elles, se dispersent. Pour les clients 0 et 1, les dix marques sont toutes différentes. Pour le client 10, une marque revient trois fois et les autres changent. La liste suit un univers de produits, au-delà d'une seule marque.

Les scores d'un même top 10 sont très proches. Le classement des derniers rangs est peu tranché. Le Recall@10, qui ne regarde que la présence dans la liste, est peu sensible à ces micro-écarts. Une métrique de rang (NDCG, par exemple) raconterait plus finement l'ordre interne de la liste. Elle n'est pas calculée dans ce pipeline.

Cet exemple porte sur trois clients. Il montre le comportement du modèle sur les premiers identifiants. Il ne mesure pas la diversité des recommandations sur les 10 383 clients évalués.

6. Choix retenu

La configuration remise pour la recommandation est **A_rank10**.

1. Son Recall@10 est 0,079953, devant 0,070010 pour le rang 20 et 0,015236 pour la popularité.
2. Elle place au moins un produit masqué dans le top 10 de 2 592 clients, contre 2 396 et 806.
3. Le protocole est contrôlé : un seul paramètre change entre A et B, la graine est fixée, le partage est un hachage du couple, et les deux tailles de cluster produisent les mêmes métriques.
4. Le temps plus court du rang 20 ne compense pas la baisse de recall. Le temps plus long avec 3 workers ne change pas la qualité.

7. Portée de la mesure

Le Recall@10 répond à une question précise : parmi les produits masqués d'un client, quelle part tombe dans les 10 suggestions nouvelles. Le rapport s'en tient à ce que le pipeline calcule.

- La métrique est une moyenne de taux de rappel. Elle décrit la couverture des achats cachés. La précision de la liste, la diversité du catalogue recommandé et la concentration sur quelques produits vedettes restent hors du JSON. La référence de popularité borne une partie de cette question : ALS dépasse nettement le classement par succès commercial.

- Le partage est un hachage de couple, reproductible d'un cluster à l'autre. Il mélange les dates. Un achat ancien peut tomber dans le test et un achat récent dans l'entraînement. Le chiffre mesure une reconstitution d'achats masqués dans la même période.
- La quantité résume l'interaction. Acheter souvent peut venir d'un renouvellement. Avec alpha = 40, une grande quantité pèse fortement dans la confiance.
- Les 2 clients sans historique et les produits absents de l'entraînement sont hors de la mesure. Une mise en production aurait besoin d'une règle de démarrage à froid (popularité, catégorie, contenu). Cette règle n'est pas dans le pipeline. Le pipeline compte ces cas, puis les écarte de l'évaluation.
- Alpha, la régularisation et le nombre d'itérations sont fixés. La comparaison ne porte que sur le rang. Un rang 20 avec une régularisation plus forte pourrait donner un autre classement des deux configurations.
- Les segments K-Means ne nourrissent pas ALS. Les deux analyses restent côte à côte dans le même job Spark, sur les mêmes transactions nettoyées.

8. Conclusion

Sur les 197 677 transactions propres, l'ALS implicite de rang 10, entraîné sur 148 083 couples client-produit, retrouve en moyenne 7,995 % des achats masqués dans une liste de 10 produits nouveaux. La même liste construite par popularité en retrouve 1,524 %. Le rang 20, à réglages égaux, en retrouve 7,001 %.

Ces trois chiffres sont les mêmes avec un worker et avec trois. La configuration à retenir est A_rank10.

Annexe — champs du JSON

Tous les effectifs et les recall de ce rapport viennent du champ **als**, identique dans les deux fichiers pour tout ce qui n'est pas une durée.

Élément du rapport	Champ
Recall et durée de A	als.configurations.A_rank10
Recall et durée de B	als.configurations.B_rank20
Configuration retenue	als.meilleure_configuration = A_rank10
Popularité	als.reference_popularite
Effectifs du partage	als.split
Exemple de listes	als.exemple_top10_meilleure_configuration
Durée de toute la partie	durations.als_seconds
Catégorie et marque de l'exemple	data/catalogue.csv

Les durées par configuration diffèrent entre **resultat_1_worker.json** et **resultat_3_workers.json**. Les recall, les nombres de clients, le partage et les 30 lignes d'exemple sont les mêmes dans les deux fichiers.